**الجمهورية الجزائرية الديمقراطية الشعبية**
**وزارة التعليم العالي والبحث العلمي**

# تقرير العمل التطبيقي (TP)

نماذج التصميم (Design Patterns) بلغة Java

**مشروع: نظام إدارة حسابات الطلبة** UniAccounts

---

العمل الأول: النواة وأنماط الإنشاء (Builder - Singleton - Prototype)

---


**الاسم واللقب:** منال يومبعي

**الفوج:** 2

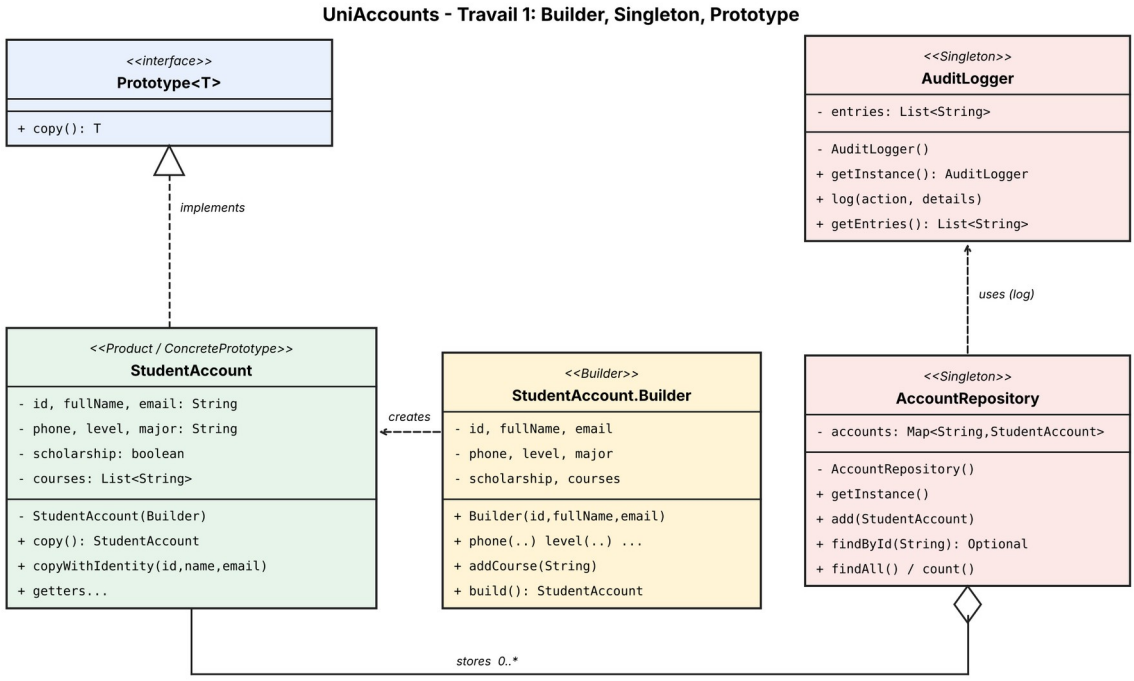
**الأستاذ المشرف:** الأستاذ يعقوب


**السنة الجامعية:** 2026 / 2027

## 1 - مقدمة

الهدف من هذا العمل بناء نواة نظام UniAccounts لإدارة حسابات الطلبة، مع استعمال ثلاثة أنماط إنشاء في الموضع الذي يحل فيه كل نمط مشكلة حقيقية: نمط Builder لبناء الحساب، ونمط Singleton لضمان وجود مستودع واحد وسجل تدقيق واحد، ونمط Prototype لإنشاء حسابات جديدة بنسخ قالب جاهز.

## 2 - مخطط الأصناف (UML)




يوضح المخطط أن `StudentAccount` ينفذ الواجهة `Prototype` (نمط Prototype)، ويبنيه الصنف المتداخل `Builder` (نمط Builder)، ويحتفظ به `AccountRepository` الذي يسجل عملياته في `AuditLogger`، وكلاهما Singleton.

**جدول ربط الأنماط بالأصناف:**

| النمط | الأصناف في الشيفرة | الدور |
|---|---|---|
| Builder | `StudentAccount.Builder` و `StudentAccount` | Builder / Product |
| Singleton | `AccountRepository` و `AuditLogger` | Singleton (مثيل واحد) |
| Prototype | `Prototype<T>` و `StudentAccount` | Prototype / ConcretePrototype |

## 3 - الأنماط المستعملة

### 3.1 نمط Builder (البنّاء)

**أين استعملته؟** في الصنف المتداخل `StudentAccount.Builder`. مُنشئ `StudentAccount` خاص (`private`)، فلا يمكن إنشاء حساب إلا عبر الـ Builder. الحقول الإجبارية الثلاثة (`id` و `fullName` و `email`) تُمرَّر إلى مُنشئ الـ Builder، والحقول الاختيارية تُضبط بدوال متسلسلة مثل `.level(..).phone(..)`. تتحقق الدالة `build()` من القواعد: id غير فارغ، وemail يحتوي على @، وlevel ضمن L1 إلى M2.

**ما المشكلة التي حلها؟** للحساب ثمانية حقول بعضها اختياري. مُنشئ واحد بثمانية وسائط غير مقروء وعرضة لأخطاء الترتيب، فمثلاً تبديل `phone` و `major` (وكلاهما String) لا يُظهر أي خطأ عند الترجمة. والـ Builder يجمع قواعد التحقق في مكان واحد فلا يخرج حساب غير صالح.

**ماذا لو لم أستعمله؟** كنت سأضطر إلى مُنشئ ضخم أو إلى عدة مُنشئات متداخلة، أو إلى setters تترك الكائن غير مكتمل أو غير صالح لفترة، وكان الكود أصعب قراءة وأكثر عرضة للأخطاء.

### 3.2 نمط Singleton (المفرد)

**أين استعملته؟** في الصنفين `AccountRepository` (المستودع المركزي) و `AuditLogger` (سجل التدقيق). في كل منهما المُنشئ خاص، والدالة `getInstance()` هي المدخل الوحيد، وتعتمد على صنف داخلي `Holder` (Initialization-on-demand holder) فيُنشأ الكائن عند أول استعمال وبشكل آمن في بيئة متعددة الخيوط. والدوال الحساسة معلَّمة `synchronized`.

**ما المشكلة التي حلها؟** يجب أن يوجد مستودع واحد يحفظ كل الحسابات وسجل تدقيق واحد. وهذا ما يسمح للمستودع برفض الـ id المكرر (قاعدة الفرادة)، وبأن تتجمع كل العمليات في سجل واحد قابل للمراجعة.

**ماذا لو لم أستعمله؟** لو أنشأ كل جزء من البرنامج مستودعه الخاص لضاعت الحسابات بين النسخ، ولم يكتشف أي مستودع أن الـ id مكرر في مستودع آخر، ولتوزّعت العمليات على سجلات متعددة يصعب تتبعها.

### 3.3 نمط Prototype (النموذج الأولي)

**أين استعملته؟** الواجهة `Prototype<T>` تفرض الدالة `copy()`، وينفذها `StudentAccount`. وأضفت الدالة `copyWithIdentity(id, name, email)` التي تنسخ القالب ثم تغيّر هوية الطالب فقط. في العرض التجريبي أنشأت قالباً لدفعة L3 تخصص Informatique بمقرراتها، ونسخته لثلاثة طلبة.

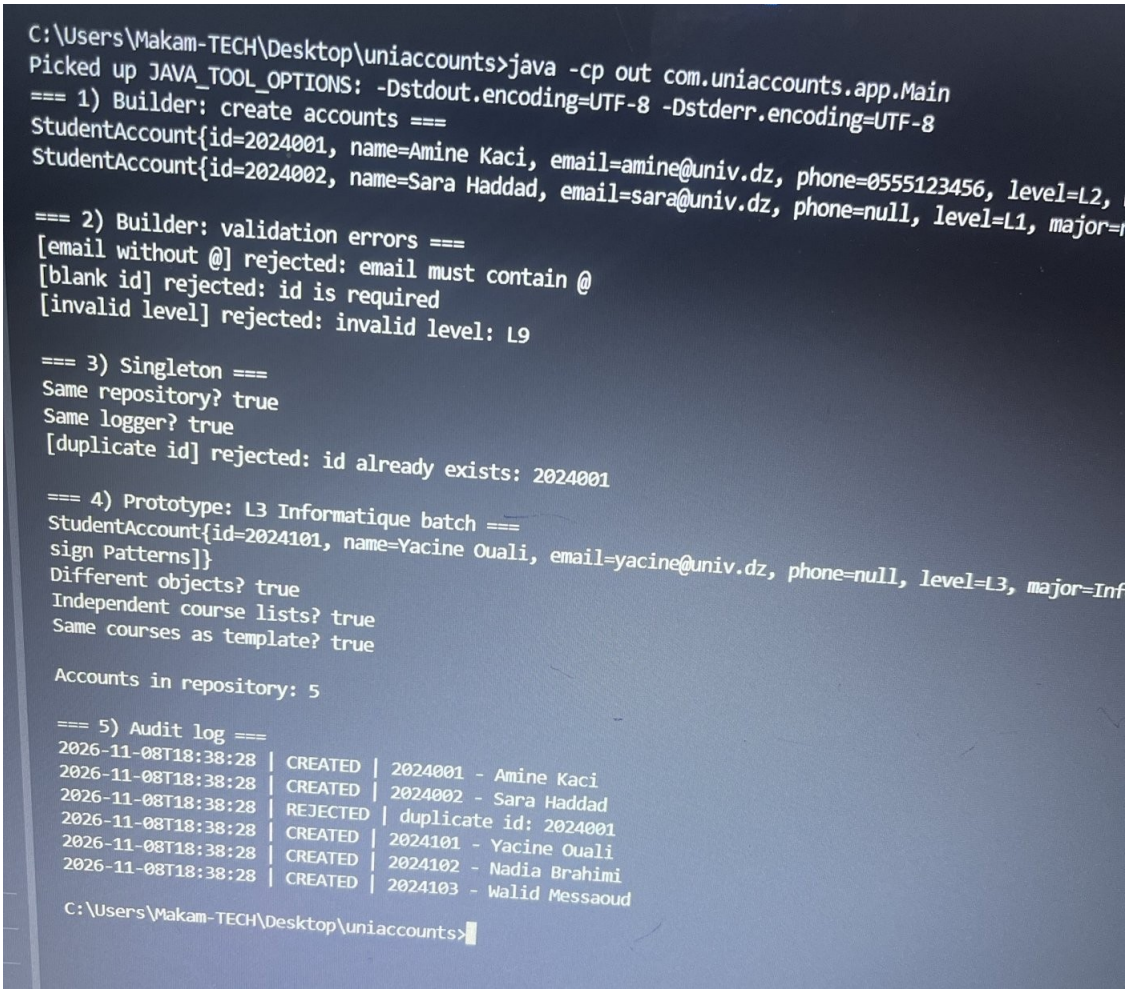
**ما المشكلة التي حلها؟** في بداية كل سنة يلتحق مئات الطلبة بنفس التخصص والمستوى والمقررات، فبدل بناء كل حساب من الصفر نحضّر قالباً للدفعة وننسخه. والنسخة **عميقة** (Deep copy): قائمة المقررات تُنسخ ولا تُشارَك، والنتائج المطبوعة تؤكد ذلك (`Independent course lists? true`).

**ماذا لو لم أستعمله؟** كنت سأكرر بناء كل حساب كاملاً بنفس القيم مئات المرات، مع خطر نسيان مقرر أو الخطأ في مستوى. ولو استعملت نسخاً سطحياً (shallow copy) لتشاركت الحسابات قائمة المقررات نفسها، فيؤثر تعديل مقرر عند طالب على بقية الطلبة.

## 4 - قرارات التصميم

| القرار | السبب |
|---|---|
| الحساب غير قابل للتعديل (كل الحقول `final`) | لا يتغير الحساب بعد إنشائه، فيبقى صالحاً دائماً. |
| النسخ يمر عبر الـ Builder | لتخضع النسخة لنفس قواعد التحقق. |
| قالب الدفعة بقيم مؤقتة (TEMPLATE) | الـ Builder يطلب الحقول الإجبارية، وهوية الطالب تُستبدل عند النسخ. |
| الحزم: `model` و `repository` و `audit` و `app` | فصل واضح بين النموذج والتخزين والسجل والعرض. |

## 5 - نتيجة التشغيل




تُظهر لقطة الشاشة الأقسام الخمسة: إنشاء الحسابات بالـ Builder، ورفض المدخلات غير الصالحة (email بلا @، و id فارغ، وlevel غير صالح)، والتحقق من أن المستودع والسجل مثيل واحد ورفض id مكرر، ونسخ قالب الدفعة ( 5 حسابات في المستودع)، وأخيراً سجل التدقيق (5 عمليات CREATED وعملية REJECTED واحدة).

## 6 - المصادر والاستعانة

استعنت بالمساعد الذكي **Claude** (من شركة Anthropic) في توليد الشيفرة الأولية وفي شرح الأنماط وإعداد مسوّدة هذا التقرير. أنشأت المشروع بنفسي في VS Code وشغّلته واختبرته على جهازي، وراجعت الشيفرة وفهمت دور كل صنف، وأتحمل مسؤولية شرح كل جزء منها في المناقشة الشفوية، وفقاً لما يسمح به نص طلب العمل (الاستعانة بالمصادر بشرط ذكرها وفهمها).

**المراجع:**


كتاب Design Patterns: Elements of Reusable Object-Oriented Software لمؤلفيه Gamma, Helm, Johnson, Vlissides.

كتاب Effective Java لمؤلفه Joshua Bloch (بنودا Builder وSingleton).

درس المقياس: نماذج التصميم (أنماط الإنشاء).